Engineering White Paper | Performance, Coverage & Sustainability Validation

Hybrid Load, Endpoint Coverage, and SQLite Concurrency Testing Methodology for SD3 Alumni

A Pre-Seeded, Two-Phase Orchestration Architecture Combining Deterministic SQL Provisioning, Fail-Fast JMeter Contract Verification, Stepped Concurrency up to 1,000 Virtual Users, Behavioral Locust Personas, and Automated Reporting

Target Environment:	**Database Engine:**	**Uplink Envelope:**	**Effective Date:**
`https://najealp.top`	SQLite (`DELETE` mode)	70 Mbps (≈ 8.75 MB/s)	November 11, 2026

Abstract

Conventional API stress tests frequently mischaracterize production readiness because of three methodological failures: silent HTTP `401 Unauthorized` cascades that inflate synthetic throughput without exercising database paths, unpaced burst loops whose samples collapse to start/end boundaries of time-series charts, and homogeneous read-only requests that overlook connection queuing and database lock contention. This white paper formalizes the deterministic load and sustainability methodology implemented for the SD3 Alumni platform (`test/load/`). Beginning with a repeatable pre-test SQL provisioning script (`test/load/hell.sql`) that snapshots SQLite `dev.db`, seeds multi-role accounts alongside moderation/RSVP targets, and normalizes credentials, the pipeline executes a fail-fast **Phase 1** pre-flight credential check and 186-sampler CRUD coverage gate (191 total requests) in Apache JMeter (`benchmark.jmx`), followed by a **Phase 2** stepped capacity sweep (100 → 1,000 VUs, run standalone via `run_jmeter.ps1` or concurrently with stateful Locust personas via `run_parallel_suite.ps1`), and concluding with **Phase 3** automated HTML, Excel, and PDF telemetry synthesis (`generate_report.py` and `export_reports_pdf.mjs`). Full five-tier empirical verification against `https://najealp.top` (`test/load/results/jmeter_report.pdf`, 17:41–17:44 local time) recorded 0.00% **HTTP failure** and 0 **HTTP `401/403/5xx` responses** across 23,157 **total requests** (191 pre-flight/coverage + 22,966 load samples, including 2 HTTP `304` cache hits), achieving an aggregate **APDEX** = 0.970 (95.06% ≤ 500 ms), overall $p_{50} = 73.00$ ms, $p_{95} = 566.00$ ms, $p_{99} = 1577.93$ ms, and peak Tier 5 throughput of 192.90 req/s (10.07 Mbps mean; 291 req/s 1-s HTML dashboard peak). Tier-by-tier telemetry confirms nearly flat response latency from 100 through 750 VUs ($p_{50} = 70$–72 ms, $p_{95} = 220$–248 ms, APDEX ≥ 0.998) after Nginx HTTP/2 keep-alive, asynchronous `bcryptjs`, in-memory ETag/TTL caching, Prisma indexing, and payload slimming optimizations, while isolating a transient TLS/socket handshake queue during the 750 → 1,000-VU ramp transition that recovers cleanly under full 1,000-VU load.

Contents

1 System Under Test and Operational Envelope **2**

1.1 Application Stack and Persistence Layer 2

1.2 Deliberate SQLite Journal Policy and Physical Bandwidth Ceiling 3

2 Methodological Foundations and Threat Model **3**

3 Two-Phase Orchestration Architecture **4**

4 Pre-Test Database Provisioning and Phase 1 Coverage Gate **5**
4.1 Pre-Test SQLite State Provisioning (`test/load/hell.sql`) . . . 5
4.2 Pre-Flight Credential Signing and Token Validation . . . 9
4.3 Comprehensive 186-Sampler Functional & CRUD Coverage Matrix . . . 9

5 Phase 2: Stepped Concurrency and Behavioral Personas **9**
5.1 Engine A: Apache JMeter Stepped Concurrency (`benchmark.jmx`) . . . 9
5.2 Engine B: Locust Headless Behavioral Personas (`locustfile.py`) . . . 10
5.2.1 Persona Pool Pre-Flight and Isolation . . . 10
5.3 Analytical Timing and APDEX Models . . . 11

6 Phase 3: Multi-Artifact Telemetry and Report Synthesis **11**
6.1 Five-Sheet Executive Workbook (`generate_report.py`) . . . 11
6.2 Automated Multi-Tab PDF Compiler (`export_reports_pdf.mjs`) . . . 12

7 Empirical Verification Results **12**
7.1 Full Five-Tier Capacity Campaign (`https://najealp.top`) . . . 12
7.2 Route-Level Performance Profile and Tier-5 Inflection Diagnosis . . . 14

8 Operational Governance and Reproducibility **15**
8.1 Database Hygiene, Snapshot Recovery, and Rate-Limiter Controls . . . 15
8.2 Standardized Execution Profiles . . . 15

1 System Under Test and Operational Envelope

1.1 Application Stack and Persistence Layer

The SD3 Alumni platform exposes a reverse-proxied monolithic Node.js backend (`api/index.js`) serving JSON REST routes and a compiled React 19 Single-Page Application (SPA) catch-all handler:

- **Reverse Proxy & Keep-Alive Envelope (`nginx/nginx.conf`, `nginx/alumni.conf`):** Public HTTPS traffic terminates at Nginx with HTTP/2 enabled (`http2 on`), `worker_rlimit_nofile 65535`, `worker_connections 4096`, `keepalive_timeout 65`, gzip compression (`gzip_comp_level 5`, `min_length 1024`), upstream connection pooling (`upstream alumni_api { keepalive 256; }`), and direct static asset serving for `/images/` and `/uploads/` with 7-day cache headers. Node.js matches this keep-alive window explicitly via `server.keepAliveTimeout = 65000` and `server.headersTimeout = 66000`.
- **HTTP Routing & JWT Authentication:** Express 5 (`express ^5.3.0`) enforces JWT authentication (`jsonwebtoken ^9.0.3`) supporting extraction from `Authorization: Bearer` headers and HTTP cookies (`token` / `adminToken`). Role separation rejects unauthenticated callers with HTTP `401` and non-user/admin mismatches with HTTP `403` via `authenticateToken` and `authenticateAdminToken`.
- **ORM, Indexing & List Payload Slimming:** Prisma ORM (`@prisma/client ^7.10.1`) binds via `@prisma/adapter-better-sqlite3` to native `better-sqlite3` (`^12.11.1`), backed by **33 explicit `@@index` declarations** in `prisma/schema.prisma`. Generic REST routes (`GET/POST/PATCH/PUT/DELETE /api/:table`) share dynamic filtering, sorting, and relational hydration via `queryParser.js` (`_where`, `_sort`, `_order`, `_include`, `_page`, `_per_page`), while

`defaultListIncludes` in `api/index.js` constrains default list-view joins so paginated JSON feeds average 12–15 KB/req (down from ≈ 85 KB/req prior to Tier-3 tuning).

- **Asynchronous Cryptography & Multi-Tier In-Memory Caching:** Password hashing and verification (`bcryptjs ^3.0.3`, cost factor 10) execute strictly asynchronously (`await bcrypt.compare(...)` and `await bcrypt.hash(..., 10)`; zero blocking `hashSync`/`compareSync` calls remain in `api/index.js`). Read hot paths leverage three bounded in-memory caches emitting weak ETags (`W/"..."`) and HTTP `304 Not Modified` responses: `feedCache` (60 s default TTL for `/api/bulletins` and `/api/events`), `lookupCache` (300 s TTL for 10 reference tables), and `adminDashboardStatsCache` (20 s TTL with single-flight `adminDashboardStatsInFlight` stampede deduplication), all invalidated automatically on mutations via `invalidateRelatedCaches`.
- **Media, Gateways & Rate Limiters:** Image uploads process through `multer` and `sharp` (`POST /api/upload`); donations integrate PayMongo (`POST /api/donations/checkout` and `/api/donations/webhook`). During benchmark campaigns, `authLimiter` and `contentLimiter` (`express-rate-limit`) bypass throttling (`skip: () => true`) to expose underlying server and transport limits.

1.2 Deliberate SQLite Journal Policy and Physical Bandwidth Ceiling

Per project architectural constraints (`api/index.js`), the database explicitly maintains rollback journal mode rather than Write-Ahead Logging:

Listing 1: Active SQLite concurrency pragmas initialized at backend startup (`api/index.js`).

```
PRAGMA journal_mode = DELETE;
PRAGMA busy_timeout = 10000;
```

In `DELETE` mode, SQLite permits concurrent shared reads when no writer commits, but acquiring an exclusive commit lock blocks all concurrent readers and writers until the rollback journal is unlinked, bounded by the 10 s adapter and SQLite `busy_timeout`. Preserving this constraint intentionally lets test suites measure lock-queue behavior realistically rather than masking it.

Simultaneously, `test/load/config.json` models the target VPS host under a physical network ceiling of $C_{\text{uplink}} = 70$ Mbits/s (≈ 8.75 MB/s). For HTTP responses with average wire payload $\bar{S}$ bytes, the theoretical network-bound request ceiling $\lambda_{\max}$ satisfies:

$$\lambda_{\max} = \frac{C_{\text{uplink}} \times 10^6}{8 \cdot \bar{S}} \quad [\text{req/s}]. \tag{1}$$

Prior to list-payload slimming, heavy paginated feeds averaging $\bar{S} \approx 85$ KB risked link saturation around 103 req/s. With Tier-3 `defaultListIncludes` slimming and Nginx gzip compression bringing paginated catalogs to 12–15 KB/req (and the blended 51-route JMeter load mix to $\bar{S} = 6.34$ KB/req), $\lambda_{\max}$ expands above 580–1,380 req/s, ensuring bandwidth does not bottleneck concurrency below 1,000 VUs.

2 Methodological Foundations and Threat Model

During suite evolution, three structural pathologies of naive load tests were identified and systematically eliminated:

1. **The Silent `401` Throughput Mirage:** If seeded accounts or JWT tokens fail before load generation, protected routes short-circuit in middleware before touching Prisma or SQLite. Throughput rises artificially while real application logic remains untested. Our methodology requires deterministic database pre-seeding (`test/load/hell.sql`) plus automated setup token verification, treating *any* `HTTP 401` across Phase 1 or Phase 2 as a hard suite failure (exit code `2`).

2. **Edge-Hugging Time-Series Collapse ("Request Slamming"):** Closed-loop threads lacking inter-request think times dispatch requests simultaneously at thread spawn and finish together before sampling intervals settle. With JMeter's default 60 s HTML report granularity and zero timer delay, time-series charts collapse to left/right edge points. Introducing randomized per-iteration `UniformRandomTimer` pacing ($\mathcal{U}(1500, 4000)$ ms in Full `config.json` defaults; $\mathcal{U}(100, 500)$ ms in early micro-benchmarks), setting `overall_granularity = 1000 ms`, and filtering setup/coverage rows out of load telemetry restores physically faithful per-second waveforms.

3. **Cross-VU State Collisions:** When multiple virtual users target identical primary keys for RSVPs, social connections, or moderation toggles, unique constraints and state-transition races distort error rates with test-harness artifacts rather than genuine server faults. We isolate state via pre-provisioned multi-account pools, VU-owned disposable resources, dedicated moderation target pools, and deterministic snapshot rollback.

Execution Modes: Standalone Stepped Capacity vs. Hybrid Read/Write Personas

The repository supports three complementary execution modes tailored to distinct engineering questions: (1) the legacy unpaced Node.js worker probe (`test/load/node_benchmark.js`) across 20 read routes for baseline socket/WAN ceilings; (2) standalone five-tier JMeter stepped capacity verification (`run_jmeter.ps1 -HtmlReport -ExcelReport -PdfReport`) driving 100 $\rightarrow$ 1,000 VUs across 51 probability-weighted routes to certify latency/APDEX scaling and publish the 4-page HTML/PDF dashboard (`jmeter_report.pdf`) plus 5-sheet Excel workbook (`jmeter_stress_test_report.xlsx`); and (3) unified parallel orchestration (`run_parallel_suite.ps1`) layering stateful write-heavy Locust personas on top of stepped JMeter load to stress `DELETE`-mode write serialization simultaneously.

3 Two-Phase Orchestration Architecture

Figure 1 illustrates the full test pipeline orchestrated from `test/load/`, enforcing Python environment validation (`.agents/python.md`), deterministic database state, fail-fast functional gating, and multi-artifact reporting.

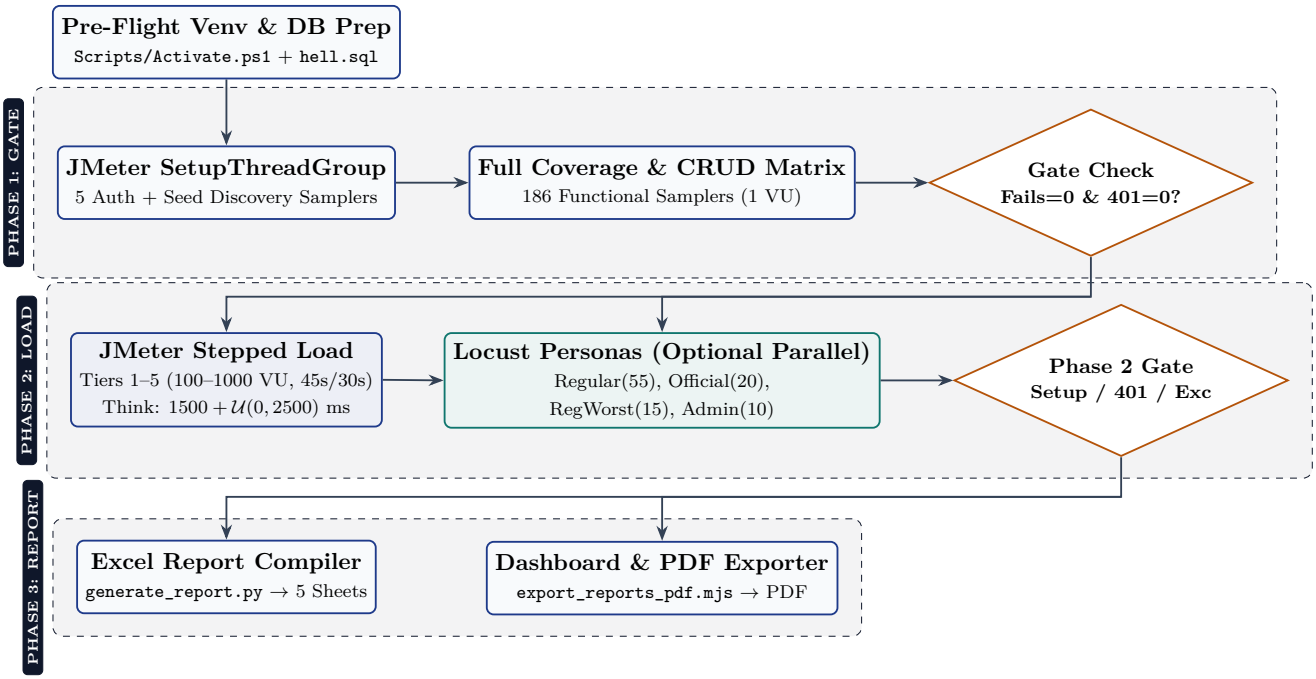


Figure 1: End-to-end orchestration pipeline across `test/load/`. Phase 0/1 prepares `dev.db` via `hell.sql` and gates Phase 2 behind a 191-sampler auth/CRUD pass; Phase 2 runs stepped JMeter load (100 → 1,000 VUs) standalone (`run_jmeter.ps1`) or alongside Locust (`run_parallel_suite.ps1`); Phase 3 generates 5-sheet Excel workbooks and Chromium-rendered PDF reports (`jmeter_report.pdf` or unified `load_test_report.pdf`).

Before invoking Python utilities or Locust, `run_parallel_suite.ps1` and `run_jmeter.ps1` validate the repository-local virtual environment: activating `Scripts/Activate.ps1`, asserting `$env:VIRTUAL_ENV` matches the repository root, verifying `python` resolves to `Scripts/python.exe`, and confirming `sys.prefix != sys.base_prefix` per `.agents/python.md`. Stale JTL, JSON, log, and HTML output artifacts in `test/load/results/` are purged prior to execution.

4 Pre-Test Database Provisioning and Phase 1 Coverage Gate

4.1 Pre-Test SQLite State Provisioning (`test/load/hell.sql`)

High-concurrency test campaigns cannot rely solely on on-the-fly API registration when multi-role connection pools (10 Administrators, 5 Official alumni, and 5 Regular alumni) must sign tokens immediately during the Phase 1 setup step without triggering registration write storms or hitting empty moderation/RSVP queues. Prior to executing the benchmark suite against `https://najealp.top`, operators execute `test/load/hell.sql` directly against the SQLite `dev.db` database on the VPS host:

1. **Pre-Stress Binary Snapshot (Step 1):** Operators take a timestamped binary copy of `dev.db` (`cp dev.db dev.db.bak_pre_stress_$(date +%s)`) prior to applying SQL mutations, establishing an instantaneous byte-exact restore point.

2. **Deterministic Multi-Role Pool Upserts (Steps 2–4):** Idempotent SQL statements seed or refresh 10 Admin accounts (`adm-loadtest-01` through `adm-loadtest-10`, usernames `admin1`–`admin10` via `ON CONFLICT(username) DO UPDATE`), 5 verified Official members (`usr-off-01`–`usr-off-05`, `userStatusId = 'OWOyGJzHfV'`), and 5 Regular members (`usr-reg-01`–`usr-reg-05`, `userStatusId`

`= '2Ue2cfwVMa'`) across `User`, `UserAuth`, public `Profile` (`kvf-XCOljU`), and `UserStatistic`, all bound to a pre-computed `bcrypt` cost-10 hash for password `123456`.

3. **Moderation & Active RSVP Targets (Step 5):** Seeds 5 Pending bulletins (`blt-pending-01`–`05`, `contentStatusId = 'a6ze5OaKs0'`), 2 Pending events (`evt-pending-01`–`02`, `eventStatusId = 'GOsRtXA38I'`), and 2 Approved RSVP events (`evt-active-rsvp-01`–`02`, `eventStatusId = 'j3Ua4WQwdl'`) using `COALESCE` category lookups so admin moderation and high-concurrency RSVP paths always locate valid targets.

4. **Global Staging Credential Normalization (Step 6):** Updates `passwordHash` across all rows in `UserAuth` and `Admin` to `$2a$10$LBJk5tBWm6IWAbNmeArmaeRHtMnDsnKsofbUbiWrbSrpMgqn5C29O` (password `123456`) so load engines can authenticate arbitrary existing staging accounts seamlessly without `401` rejections. Because this step irreversibly overwrites prior password digests, Step 1 backup capture and post-test restoration are mandatory.

Listing 2 reproduces the complete pre-test SQL initialization script executed before the benchmark campaign.

Listing 2: Complete pre-test SQLite setup script (`test/load/hell.sql`) executed prior to load testing.

```sql
-- ============================================================================
-- PRE-TEST DATABASE SETUP SCRIPT FOR HIGH-CONCURRENCY LOAD TESTING
-- Target: https://najealp.top (dev.db SQLite Database)
-- Password for all generated test accounts: 123456
-- Bcrypt Hash (cost 10): $2b$10$OWJFhm7ALzwkqTNSmEOqYOOhwTnHlGLAG8s5qAMmfqvhHozxa4ZzW
-- ============================================================================
--
-- ----------------------------------------------------------------------------
-- STEP 1: VPS SAFETY BACKUP
-- Run this in bash on the VPS BEFORE running this SQL file:
--   cp dev.db dev.db.bak_pre_stress_$(date +%s)
-- ----------------------------------------------------------------------------

-- ----------------------------------------------------------------------------
-- STEP 2: PROVISION 10 DEDICATED ADMINISTRATORS
-- Usernames: admin1 through admin10 | Password: "123456"
-- ----------------------------------------------------------------------------
INSERT INTO Admin (id, username, passwordHash, lastLogin)
VALUES
    ('adm-loadtest-01', 'admin1', '$2b$10$OWJFhm7ALzwkqTNSmEOqYOOhwTnHlGLAG8s5qAMmfqvhHozxa4ZzW', CURRENT_TIMESTAMP),
    ('adm-loadtest-02', 'admin2', '$2b$10$OWJFhm7ALzwkqTNSmEOqYOOhwTnHlGLAG8s5qAMmfqvhHozxa4ZzW', CURRENT_TIMESTAMP),
    ('adm-loadtest-03', 'admin3', '$2b$10$OWJFhm7ALzwkqTNSmEOqYOOhwTnHlGLAG8s5qAMmfqvhHozxa4ZzW', CURRENT_TIMESTAMP),
    ('adm-loadtest-04', 'admin4', '$2b$10$OWJFhm7ALzwkqTNSmEOqYOOhwTnHlGLAG8s5qAMmfqvhHozxa4ZzW', CURRENT_TIMESTAMP),
    ('adm-loadtest-05', 'admin5', '$2b$10$OWJFhm7ALzwkqTNSmEOqYOOhwTnHlGLAG8s5qAMmfqvhHozxa4ZzW', CURRENT_TIMESTAMP),
    ('adm-loadtest-06', 'admin6', '$2b$10$OWJFhm7ALzwkqTNSmEOqYOOhwTnHlGLAG8s5qAMmfqvhHozxa4ZzW', CURRENT_TIMESTAMP),
    ('adm-loadtest-07', 'admin7', '$2b$10$OWJFhm7ALzwkqTNSmEOqYOOhwTnHlGLAG8s5qAMmfqvhHozxa4ZzW', CURRENT_TIMESTAMP),
    ('adm-loadtest-08', 'admin8', '$2b$10$OWJFhm7ALzwkqTNSmEOqYOOhwTnHlGLAG8s5qAMmfqvhHozxa4ZzW', CURRENT_TIMESTAMP),
    ('adm-loadtest-09', 'admin9', '$2b$10$OWJFhm7ALzwkqTNSmEOqYOOhwTnHlGLAG8s5qAMmfqvhHozxa4ZzW', CURRENT_TIMESTAMP),
    ('adm-loadtest-10', 'admin10', '$2b$10$OWJFhm7ALzwkqTNSmEOqYOOhwTnHlGLAG8s5qAMmfqvhHozxa4ZzW', CURRENT_TIMESTAMP)
ON CONFLICT(username) DO UPDATE SET
    passwordHash = excluded.passwordHash,
    lastLogin = CURRENT_TIMESTAMP;

-- ----------------------------------------------------------------------------
-- STEP 3: PROVISION 5 VERIFIED OFFICIAL TEST USERS
-- Emails: test.official1@najealp.top to test.official5@najealp.top | Password: "123456"
-- Status: Official ('OWOyGJzHfV') | Profile: Public ('kvf-XCOljU')
-- ----------------------------------------------------------------------------

-- User 1 (Official)
INSERT OR REPLACE INTO User (id, userStatusId, profileStatusId)
VALUES ('usr-off-01', 'OWOyGJzHfV', 'kvf-XCOljU');
INSERT OR REPLACE INTO UserAuth (userId, email, passwordHash, lastLogin)
VALUES ('usr-off-01', 'test.official1@najealp.top', '$2b$10$OWJFhm7ALzwkqTNSmEOqYOOhwTnHlGLAG8s5qAMmfqvhHozxa4ZzW',
    CURRENT_TIMESTAMP);
INSERT OR REPLACE INTO Profile (userId, userName, gender, bio, email, degreeId, batch)
VALUES ('usr-off-01', 'Official Tester One', 'Female', 'Verified Official Load Test Account', 'test.official1@najealp.
    top', 'hd2F5ini8q', 2022);
INSERT OR REPLACE INTO UserStatistic (userId, userConnections, eventsAttended, eventsCreated, bulletinsCreated,
    commentsWritten, achievements, donatedAmount, dateRegistered)
VALUES ('usr-off-01', 5, 2, 5, 3, 10, 2, 500.0, CURRENT_TIMESTAMP);

-- User 2 (Official)
INSERT OR REPLACE INTO User (id, userStatusId, profileStatusId)
VALUES ('usr-off-02', 'OWOyGJzHfV', 'kvf-XCOljU');
INSERT OR REPLACE INTO UserAuth (userId, email, passwordHash, lastLogin)
```

```
VALUES ('usr-off-02', 'test.official2@najealp.top', '$2b$10$OWJFhm7ALzwkqTNSmE0qYO0hwTnHlGLAG8s5qAMmfqvhHozxa4ZzW',
    CURRENT_TIMESTAMP);
INSERT OR REPLACE INTO Profile (userId, userName, gender, bio, email, degreeId, batch)
VALUES ('usr-off-02', 'Official Tester Two', 'Male', 'Verified Official Load Test Account', 'test.official2@najealp.
    top', 'GISquVZHHs', 2021);
INSERT OR REPLACE INTO UserStatistic (userId, userConnections, eventsAttended, eventsCreated, bulletinsCreated,
    commentsWritten, achievements, donatedAmount, dateRegistered)
VALUES ('usr-off-02', 4, 1, 3, 2, 8, 1, 250.0, CURRENT_TIMESTAMP);

-- User 3 (Official)
INSERT OR REPLACE INTO User (id, userStatusId, profileStatusId)
VALUES ('usr-off-03', 'OWOyGJzHfV', 'kvf-XCOljU');
INSERT OR REPLACE INTO UserAuth (userId, email, passwordHash, lastLogin)
VALUES ('usr-off-03', 'test.official3@najealp.top', '$2b$10$OWJFhm7ALzwkqTNSmE0qYO0hwTnHlGLAG8s5qAMmfqvhHozxa4ZzW',
    CURRENT_TIMESTAMP);
INSERT OR REPLACE INTO Profile (userId, userName, gender, bio, email, degreeId, batch)
VALUES ('usr-off-03', 'Official Tester Three', 'Female', 'Verified Official Load Test Account', 'test.
    official3@najealp.top', 'AUG-4vBe6k', 2023);
INSERT OR REPLACE INTO UserStatistic (userId, userConnections, eventsAttended, eventsCreated, bulletinsCreated,
    commentsWritten, achievements, donatedAmount, dateRegistered)
VALUES ('usr-off-03', 6, 3, 4, 4, 15, 3, 1000.0, CURRENT_TIMESTAMP);

-- User 4 (Official)
INSERT OR REPLACE INTO User (id, userStatusId, profileStatusId)
VALUES ('usr-off-04', 'OWOyGJzHfV', 'kvf-XCOljU');
INSERT OR REPLACE INTO UserAuth (userId, email, passwordHash, lastLogin)
VALUES ('usr-off-04', 'test.official4@najealp.top', '$2b$10$OWJFhm7ALzwkqTNSmE0qYO0hwTnHlGLAG8s5qAMmfqvhHozxa4ZzW',
    CURRENT_TIMESTAMP);
INSERT OR REPLACE INTO Profile (userId, userName, gender, bio, email, degreeId, batch)
VALUES ('usr-off-04', 'Official Tester Four', 'Male', 'Verified Official Load Test Account', 'test.official4@najealp.
    top', 'YZcAIyFl6L', 2020);
INSERT OR REPLACE INTO UserStatistic (userId, userConnections, eventsAttended, eventsCreated, bulletinsCreated,
    commentsWritten, achievements, donatedAmount, dateRegistered)
VALUES ('usr-off-04', 3, 2, 2, 1, 5, 1, 100.0, CURRENT_TIMESTAMP);

-- User 5 (Official)
INSERT OR REPLACE INTO User (id, userStatusId, profileStatusId)
VALUES ('usr-off-05', 'OWOyGJzHfV', 'kvf-XCOljU');
INSERT OR REPLACE INTO UserAuth (userId, email, passwordHash, lastLogin)
VALUES ('usr-off-05', 'test.official5@najealp.top', '$2b$10$OWJFhm7ALzwkqTNSmE0qYO0hwTnHlGLAG8s5qAMmfqvhHozxa4ZzW',
    CURRENT_TIMESTAMP);
INSERT OR REPLACE INTO Profile (userId, userName, gender, bio, email, degreeId, batch)
VALUES ('usr-off-05', 'Official Tester Five', 'Female', 'Verified Official Load Test Account', 'test.official5@najealp
    .top', 'hd2F5ini8q', 2024);
INSERT OR REPLACE INTO UserStatistic (userId, userConnections, eventsAttended, eventsCreated, bulletinsCreated,
    commentsWritten, achievements, donatedAmount, dateRegistered)
VALUES ('usr-off-05', 8, 4, 6, 5, 20, 4, 1500.0, CURRENT_TIMESTAMP);

-- ----------------------------------------------------------------------------
-- STEP 4: PROVISION 5 REGULAR TEST USERS
-- Emails: test.regular1@najealp.top to test.regular5@najealp.top | Password: "123456"
-- Status: Regular ('2Ue2cfwVMa') | Profile: Public ('kvf-XCOljU')
-- ----------------------------------------------------------------------------

-- Regular 1
INSERT OR REPLACE INTO User (id, userStatusId, profileStatusId)
VALUES ('usr-reg-01', '2Ue2cfwVMa', 'kvf-XCOljU');
INSERT OR REPLACE INTO UserAuth (userId, email, passwordHash, lastLogin)
VALUES ('usr-reg-01', 'test.regular1@najealp.top', '$2b$10$OWJFhm7ALzwkqTNSmE0qYO0hwTnHlGLAG8s5qAMmfqvhHozxa4ZzW',
    CURRENT_TIMESTAMP);
INSERT OR REPLACE INTO Profile (userId, userName, gender, bio, email, degreeId, batch)
VALUES ('usr-reg-01', 'Regular Tester One', 'Male', 'Regular Test Member', 'test.regular1@najealp.top', 'hd2F5ini8q',
    2024);
INSERT OR REPLACE INTO UserStatistic (userId, userConnections, eventsAttended, eventsCreated, bulletinsCreated,
    commentsWritten, achievements, donatedAmount, dateRegistered)
VALUES ('usr-reg-01', 1, 0, 0, 0, 2, 0, 0.0, CURRENT_TIMESTAMP);

-- Regular 2
INSERT OR REPLACE INTO User (id, userStatusId, profileStatusId)
VALUES ('usr-reg-02', '2Ue2cfwVMa', 'kvf-XCOljU');
INSERT OR REPLACE INTO UserAuth (userId, email, passwordHash, lastLogin)
VALUES ('usr-reg-02', 'test.regular2@najealp.top', '$2b$10$OWJFhm7ALzwkqTNSmE0qYO0hwTnHlGLAG8s5qAMmfqvhHozxa4ZzW',
    CURRENT_TIMESTAMP);
INSERT OR REPLACE INTO Profile (userId, userName, gender, bio, email, degreeId, batch)
VALUES ('usr-reg-02', 'Regular Tester Two', 'Female', 'Regular Test Member', 'test.regular2@najealp.top', 'N3MmjKro9B'
    , 2023);
INSERT OR REPLACE INTO UserStatistic (userId, userConnections, eventsAttended, eventsCreated, bulletinsCreated,
    commentsWritten, achievements, donatedAmount, dateRegistered)
VALUES ('usr-reg-02', 2, 1, 0, 1, 4, 0, 0.0, CURRENT_TIMESTAMP);

-- Regular 3
INSERT OR REPLACE INTO User (id, userStatusId, profileStatusId)
VALUES ('usr-reg-03', '2Ue2cfwVMa', 'kvf-XCOljU');
INSERT OR REPLACE INTO UserAuth (userId, email, passwordHash, lastLogin)
VALUES ('usr-reg-03', 'test.regular3@najealp.top', '$2b$10$OWJFhm7ALzwkqTNSmE0qYO0hwTnHlGLAG8s5qAMmfqvhHozxa4ZzW',
    CURRENT_TIMESTAMP);
INSERT OR REPLACE INTO Profile (userId, userName, gender, bio, email, degreeId, batch)
```

```
VALUES ('usr-reg-03', 'Regular Tester Three', 'Male', 'Regular Test Member', 'test.regular3@najealp.top', 'hAoJF3hX0R', 2022);
INSERT OR REPLACE INTO UserStatistic (userId, userConnections, eventsAttended, eventsCreated, bulletinsCreated, commentsWritten, achievements, donatedAmount, dateRegistered)
VALUES ('usr-reg-03', 0, 0, 0, 0, 1, 0, 0.0, CURRENT_TIMESTAMP);

-- Regular 4
INSERT OR REPLACE INTO User (id, userStatusId, profileStatusId)
VALUES ('usr-reg-04', '2Ue2cfwVMa', 'kvf-XC0ljU');
INSERT OR REPLACE INTO UserAuth (userId, email, passwordHash, lastLogin)
VALUES ('usr-reg-04', 'test.regular4@najealp.top', '$2b$10$OWJFhm7ALzwkqTNSmE0qYO0hwTnHlGLAG8s5qAMmfqvhHozxa4ZzW', CURRENT_TIMESTAMP);
INSERT OR REPLACE INTO Profile (userId, userName, gender, bio, email, degreeId, batch)
VALUES ('usr-reg-04', 'Regular Tester Four', 'Female', 'Regular Test Member', 'test.regular4@najealp.top', 'GISquVZHHs', 2025);
INSERT OR REPLACE INTO UserStatistic (userId, userConnections, eventsAttended, eventsCreated, bulletinsCreated, commentsWritten, achievements, donatedAmount, dateRegistered)
VALUES ('usr-reg-04', 1, 0, 0, 0, 0, 0, 0.0, CURRENT_TIMESTAMP);

-- Regular 5
INSERT OR REPLACE INTO User (id, userStatusId, profileStatusId)
VALUES ('usr-reg-05', '2Ue2cfwVMa', 'kvf-XC0ljU');
INSERT OR REPLACE INTO UserAuth (userId, email, passwordHash, lastLogin)
VALUES ('usr-reg-05', 'test.regular5@najealp.top', '$2b$10$OWJFhm7ALzwkqTNSmE0qYO0hwTnHlGLAG8s5qAMmfqvhHozxa4ZzW', CURRENT_TIMESTAMP);
INSERT OR REPLACE INTO Profile (userId, userName, gender, bio, email, degreeId, batch)
VALUES ('usr-reg-05', 'Regular Tester Five', 'Male', 'Regular Test Member', 'test.regular5@najealp.top', 'hd2F5ini8q', 2021);
INSERT OR REPLACE INTO UserStatistic (userId, userConnections, eventsAttended, eventsCreated, bulletinsCreated, commentsWritten, achievements, donatedAmount, dateRegistered)
VALUES ('usr-reg-05', 3, 1, 0, 1, 3, 1, 50.0, CURRENT_TIMESTAMP);

-- ------------------------------------------------------------------------------
-- STEP 5: PROVISION PENDING & APPROVED ITEMS FOR MODERATION TESTING
-- Creates 5 pending bulletins and 3 pending events for admins to approve/reject
-- ------------------------------------------------------------------------------

-- Pending Bulletins ('a6ze5OaKs0')
-- Bulletin Categories: Announcements ('ueMRjAn-P0'), Careers ('nEA2WEPc5f'), Donations ('xd67TtDe0d')
INSERT OR REPLACE INTO Bulletin (id, title, content, contentStatusId, authorId, bulletinDate, bulletinCategoryId, readTimeMinutes)
VALUES
    ('blt-pending-01', 'Alumni Homecoming Planning 2026', 'Draft agenda for the upcoming grand homecoming banquet and symposium.', 'a6ze5OaKs0', 'usr-off-01', CURRENT_TIMESTAMP, COALESCE((SELECT id FROM BulletinCategory WHERE bulletinCategoryName = 'Announcements' LIMIT 1), 'ueMRjAn-P0'), 3),
    ('blt-pending-02', 'Mentorship Program Signups', 'Seeking senior engineers to mentor graduating batch of 2026.', 'a6ze5OaKs0', 'usr-off-02', CURRENT_TIMESTAMP, COALESCE((SELECT id FROM BulletinCategory WHERE bulletinCategoryName = 'Careers' LIMIT 1), 'nEA2WEPc5f'), 2),
    ('blt-pending-03', 'Robotics Lab Equipment Fund', 'Call for donations to upgrade university microcontroller kits.', 'a6ze5OaKs0', 'usr-reg-02', CURRENT_TIMESTAMP, COALESCE((SELECT id FROM BulletinCategory WHERE bulletinCategoryName = 'Donations' LIMIT 1), 'xd67TtDe0d'), 4),
    ('blt-pending-04', 'Start-up Pitch Night Cebu', 'Calling all alumni founders to pitch in front of angel investors.', 'a6ze5OaKs0', 'usr-off-03', CURRENT_TIMESTAMP, COALESCE((SELECT id FROM BulletinCategory WHERE bulletinCategoryName = 'Careers' LIMIT 1), 'nEA2WEPc5f'), 3),
    ('blt-pending-05', 'Career Transition AMA', 'Join us next week for an AMA on transitioning from hardware to cloud dev.', 'a6ze5OaKs0', 'usr-reg-05', CURRENT_TIMESTAMP, COALESCE((SELECT id FROM BulletinCategory WHERE bulletinCategoryName = 'Careers' LIMIT 1), 'nEA2WEPc5f'), 5);

-- Pending Events ('G0sRtXA38I') & Approved Events for RSVP ('j3Ua4WQwdl')
-- Event Categories: Reunion ('lkIFfpktVL'), Conference ('iH-gzGZDnh'), Networking ('d3Kg5La8vg'), Workshop ('73JAuNaAoA')
INSERT OR REPLACE INTO Event (id, title, description, eventStatusId, authorId, eventDate, modality, eventCategoryId)
VALUES
    ('evt-pending-01', 'Annual General Assembly 2026', 'Official election of new alumni board officers and annual report.', 'G0sRtXA38I', 'usr-off-01', '2026-12-15 09:00:00', 'In-Person', COALESCE((SELECT id FROM EventCategory WHERE eventCategoryName = 'Reunion' LIMIT 1), 'lkIFfpktVL')),
    ('evt-pending-02', 'Tech Leadership Panel', 'A keynote panel discussing AI agents and sustainable computing.', 'G0sRtXA38I', 'usr-off-02', '2026-11-25 14:00:00', 'Virtual', COALESCE((SELECT id FROM EventCategory WHERE eventCategoryName = 'Conference' LIMIT 1), 'iH-gzGZDnh')),
    ('evt-active-rsvp-01', 'High Concurrency Mega Meetup', 'Active event receiving massive RSVPs from Regular & Official members.', 'j3Ua4WQwdl', 'usr-off-01', '2026-11-30 18:00:00', 'In-Person', COALESCE((SELECT id FROM EventCategory WHERE eventCategoryName = 'Networking' LIMIT 1), 'd3Kg5La8vg')),
    ('evt-active-rsvp-02', 'Alumni Hackathon 2026', 'Active event receiving massive RSVPs to test event concluding workflow.', 'j3Ua4WQwdl', 'usr-off-03', '2026-11-28 10:00:00', 'Virtual', COALESCE((SELECT id FROM EventCategory WHERE eventCategoryName = 'Workshop' LIMIT 1), '73JAuNaAoA'));

-- THIS CANNOT BE UNDONE!!! Satan has leaked all the secrets! The password of every user is 123456.
-- BACKUP THE DATABASE NOW!!! LAST WARNING!!!
UPDATE UserAuth
SET passwordHash = '$2a$10$LBJk5tBWm6IWAbNmeArmaeRHtMnDsnKsofbUbiWrbSrpMgqn5C29O'
;

UPDATE Admin
SET passwordHash = '$2a$10$LBJk5tBWm6IWAbNmeArmaeRHtMnDsnKsofbUbiWrbSrpMgqn5C29O'
;
```

4.2 Pre-Flight Credential Signing and Token Validation

Because `benchmark.jmx` sets `TestPlan.serialize_threadgroups=true`, its `SetupThreadGroup` executes strictly before any subsequent thread group. It contains five samplers:

1. A Groovy `JSR223Sampler` (`[Setup] Sign Credentials, Verify All Tokens & Discover Seed IDs`) that reads candidate accounts from `config.json` (default password `123456`), authenticates Admin, Official, and Regular accounts, provisions missing functional testers via `/api/admin/users` or `/api/auth/reset-password` when absent, and extracts live entity IDs (`sampleBulletinId`, `sampleEventId`, status lookup IDs, categories, degrees) into global JMeter properties.
2. Four explicit HTTP verification samplers validate the minted tokens end-to-end before Phase 1 proceeds: `GET /api/auth/admin/me`, `GET /api/admin/dashboard-stats`, `GET /api/auth/me (Official)`, and `GET /api/auth/me (Regular)`.

4.3 Comprehensive 186-Sampler Functional & CRUD Coverage Matrix

Immediately following setup verification, the `Full Endpoint Coverage & CRUD Matrix` thread group runs as a single serialized thread (`-CoverageOnly` sets stepped tier threads to zero; `-SkipCoverage` sets coverage threads to zero) exercising **186 functional requests** (191 total including the 5 setup samplers):

- **Lookup & Relational Catalog Reads:** All read-only status dictionaries (`userStatuses`, `contentStatuses`, `eventStatuses`, `donationStatuses`, `profileStatuses`, `connectionStatuses`, `degrees`, categories, achievements), `HEAD` requests, paginated feeds, `_include` relational joins, and `_sort`/`_order` ordering checks.
- **Query Parser Operator Coverage:** Targeted probes on `/api/achievements` and `/api/users` verifying `_where` predicates including `contains`, `endsWith`, `startsWith`, `gt`, combined `in+gte+lte`, `notIn`, and nested foreign-key filtering.
- **Isolated CRUD Lifecycles (`POST` → `GET` → `PATCH` → `PUT` → `DELETE`):** Round-trip creation, retrieval, partial/full updates, and deterministic deletion across domain tables (`locations`, `bulletins`, `bulletinLikes`, `comments`, `commentLikes`, `events`, `userRsvps`, `donations`, `clefLogs`, `records`, `notifications`, `userAchievements`, plus admin user/bulletin/event creation).
- **Stateful Workflows, Media & External Gateways:** Multipart fixture upload (`fixture.png` → `POST /api/upload` → `GET /uploads/...`), SPA root and hero asset checks, disposable password-reset verification, all social connection transitions on `/api/connections/action` (`add`, `accept`, `reject`, `remove`, `block`, `unblock`), event conclusion (`POST /api/events/:id/conclude`), and conditional PayMongo checkout/webhook tests toggled by `INCLUDE_EXTERNAL_APIS`.

If `total401 > 0` or `totalFailed > 0`, the runner logs offending route breakdowns and aborts immediately without proceeding to Phase 2 reporting.

5 Phase 2: Stepped Concurrency and Behavioral Personas

Phase 2 scales stepped concurrency across up to five sequential thread-group tiers in JMeter—executed either standalone via `run_jmeter.ps1` (as in the full 5-tier campaign analyzed in Section 7) or concurrently alongside Locust behavioral personas via `run_parallel_suite.ps1`.

5.1 Engine A: Apache JMeter Stepped Concurrency (`benchmark.jmx`)

Whether executed directly after the 191-sampler coverage gate or isolated via `-SkipCoverage`, five sequential stepped thread groups (`TIER1`–`TIER5`) invoke a shared `ModuleController` referencing `Load`

`Mix Test Fragment`:

- **Tier Profiles:** `DryRun` evaluates $V \in \{10, 25\}$ VUs for $D_{\text{tier}} = 10$ s each ($R_{\text{ramp}} = 2$ s); `Full` defaults per `test/load/config.json` to $V \in \{100, 250, 500, 750, 1000\}$ VUs for $D_{\text{tier}} = 45$ s each with linear thread ramp-up $R_{\text{ramp}} = 30$ s. Thread groups assigned 0 threads are skipped.
- **Probability-Weighted 51-Sampler Dispatch & HTTP Caching:** Inside `Load Mix Test Fragment`, an `HTTP Cache Manager` (`clearEachIteration=false`, `useExpires=true`) preserves conditional validators (`ETag`/`Cache-Control`), while a Groovy-backed `SwitchController` routes iterations across four weighted `RandomController` buckets totaling **51 enabled samplers** (plus optional CRUD writes gated by `-IncludeWritesInLoad`):
  1. *Bucket 0 — Feeds & Public (*55%*,* 13 *samplers,* $\approx$ 4.23% *each):* Paginated/sorted/hydrated `/api/bulletins` and `/api/events` views, detail routes, `/api/server-time`, `/api/hero/images`, static `/images/engineer.png`, and `HEAD /api/bulletins`.
  2. *Bucket 1 — User & Social (*30%*,* 20 *samplers,* $\approx$ 1.50% *each):* Paginated `/api/users` and filtered `_where` queries, `/api/auth/me` (Regular & Official), `/api/donations/summary`, `/api/userConnections`, `/api/comments`, `/api/notifications`, `/api/userRsvps`, `POST /api/auth/check-email`, and profile/achievement tables.
  3. *Bucket 2 — Reference Lookups (*11%*,* 11 *samplers,* 1.00% *each):* Status dictionaries, degrees, categories, and achievements.
  4. *Bucket 3 — Admin & Audit (*4%*,* 7 *samplers,* $\approx$ 0.57% *each):* `/api/admin/dashboard-stats`, `/api/admin/bulletins`, `/api/admin/events`, `/api/auth/admin/me`, `/api/donations`, `/api/clefLogs`, and `/api/userAuths`.
- **Human-Realistic Think-Time Pacing:** Every iteration pauses via `UniformRandomTimer` parameterized from `config.json` (`defaultThinkMinMs = 1500`, `defaultThinkRangeMs = 2500`):

$$T_{\text{think}} \sim \mathcal{U}(t_{\min},\, t_{\min} + \Delta t) = \mathcal{U}(1500,\, 4000) \text{ ms}, \qquad \mathbb{E}[T_{\text{think}}] = 2750 \text{ ms}. \tag{2}$$

- **Clean Load-Only Dashboard Filtering:** Prior to invoking JMeter's HTML ReportGenerator at `overall_granularity = 1000 ms`, `run_jmeter.ps1` filters the raw JTL stream with a *BOM-free UTF-8* writer (`[System.Text.UTF8Encoding]::new($false)`) retaining only rows matching `^Stepped Tier \d+` into `jmeter_results_load.jtl`. Removing the UTF-8 Byte Order Mark prevents JMeter CSV parser header corruption, while excluding Setup/Coverage rows keeps initial auth warmup outliers out of load APDEX and percentiles.

5.2 Engine B: Locust Headless Behavioral Personas (`locustfile.py`)

When invoked via `run_parallel_suite.ps1`, Locust executes concurrently alongside JMeter for $T_{\text{locust}} = \max(15,\, N_{\text{active_tiers}} \cdot D_{\text{tier}} + 4)$ s (24 s in DryRun at 20 VUs spawned at 5 VU/s; 100 VUs at 10 VU/s in Full defaults). To prevent Windows headless `Start-Process` stdin errors in `locust/input_events.py` (`isatty()` on a null handle), `run_parallel_suite.ps1` binds standard input to an empty temporary file cleaned up on exit.

5.2.1 Persona Pool Pre-Flight and Isolation

In `on_test_start`, Locust validates Admin, Official (prioritizing `loadtest_jmx_official@najealp.top`), and Regular pools in parallel via `ThreadPoolExecutor(max_workers=10)`, automatically provisioning missing testers when needed and aborting with `SETUP_ERROR` if any role pool

remains empty. Pre-flight seeds 5 disposable Pending bulletins plus 4 disposable events (2 moderation targets, 2 conclude targets).

Table 1: Behavioral workload distribution and pacing parameters in `test/load/locustfile.py`.

Persona Class	Weight	Wait Time (s)	Primary Stateful Workflow & Isolation Strategy
`RegularUser`	55%	$\mathcal{U}(0.5, 2.0)$	Browses sorted/filtered bulletins & events, relational directories, lookups; creates VU-exclusive event (`own_event_id`) + toggles private RSVP (`own_rsvp_id`); patches own profile bio; posts tracked community bulletins.
`OfficialUser`	20%	$\mathcal{U}(0.3, 1.5)$	Posts approved official bulletins ($w = 4$); manages exclusive official event RSVP lifecycle ($w = 3$); inspects relational bulletin catalogs ($w = 2$).
`RegistrationWorstCase`	15%	$\mathcal{U}(0.8, 2.5)$	Sends `POST /api/auth/register` bursts combining cost-10 `bcrypt` key derivation with multi-table SQLite writes.
`CrazyAdmin`	10%	$\mathcal{U}(0.1, 0.5)$	Aggressively polls `/api/admin/dashboard-stats` and donation summaries/tables while flipping moderation statuses across disposable seeded bulletin/event targets.

Each Locust user validates its token in `on_start` (`/api/auth/me` or `/api/auth/admin/me`), cleans up its VU-owned RSVP and event in `on_stop`, and relies on `on_test_stop` to purge all `CREATED_BULLETIN_IDS` and `CREATED_EVENT_IDS`, serialize `locust_metrics.json`, and enforce gates on `setupError`, `count401`, `totalErrors`, and `exceptionsCount`.

5.3 Analytical Timing and APDEX Models

For closed-loop paced virtual users of count V experiencing mean server response latency $\bar{L}_{\text{resp}}$ and expected think time $\mathbb{E}[T_{\text{think}}]$, Little's Law predicts full-concurrency steady-state throughput:

$$\lambda(V) = \frac{V}{\bar{L}_{\text{resp}} + \mathbb{E}[T_{\text{think}}]}. \tag{3}$$

Over a tier duration D_{tier} containing a linear thread ramp-up $R_{\text{ramp}} \leq D_{\text{tier}}$, effective mean active concurrency is scaled approximately by $(D_{\text{tier}} - \frac{1}{2}R_{\text{ramp}})/D_{\text{tier}}$. User-perceived responsiveness is quantified via JMeter's standard Application Performance Index (APDEX) with thresholds $T = 500$ ms (satisfied) and $F = 1500$ ms (frustrated):

$$\text{APDEX} = \frac{N_{t \leq T} + 0.5 \cdot N_{T < t \leq F}}{N_{\text{total}}}. \tag{4}$$

6 Phase 3: Multi-Artifact Telemetry and Report Synthesis

6.1 Five-Sheet Executive Workbook (`generate_report.py`)

Configured via `--source` (`auto`, `jmeter`, or `node`) and optional `--no-locust`, `generate_report.py` compiles `docs/stress_test_report.xlsx` (or `docs/jmeter_stress_test_report.xlsx` in standalone JMeter mode via `run_jmeter.ps1 -ExcelReport`):

1. **Executive Summary & Verdict:** KPI cards (requests, peak RPS, Mbps, error rate, $p_{50}/p_{95}/p_{99}$, % of 70 Mbps link) plus narrative analysis contrasting SQLite's fast in-process reads against `DELETE`-mode write locking.

2. **Latency Evolution & Scaling:** Per-tier concurrency metrics paired with a native Excel line chart tracking p_{50}, p_{95}, and p_{99} across stepped VU tiers.
3. **Transaction Rankings:** Routes ordered by cumulative time consumption (`totalTimeMs`) and tagged into *Standard Read*, *Relational Hydration*, *Filtered & Sorted*, and *Admin Aggregation*.
4. **Transaction Variances:** Top-5 HTTP status code distributions per route with per-status min/median/max latencies and error snippets.
5. **Phase / Baseline Comparison:** In hybrid mode (*Baseline vs. Personas*), contrasts JMeter/Node stepped load with Locust persona timings; with `--no-locust` (*JMeter Phase Breakdown*), reports Setup (5), Coverage (186), and Stepped Tier 1–5 totals independently.

6.2 Automated Multi-Tab PDF Compiler (`export_reports_pdf.mjs`)

To compile JMeter's interactive Flot HTML dashboard (and, when active, Locust's ECharts/-MUI dashboard) into self-contained PDF reports (`jmeter_report.pdf` under `--jmeter-only` or `load_test_report.pdf` in combined runs), `export_reports_pdf.mjs` serves `test/load/results/` over an ephemeral HTTP server driven by headless Edge/Chrome via `puppeteer-core`:

- **Iframe Isolation & Anchor-Navigation Bypass:** Each dashboard tab (`index.html`, `OverTime.html`, `Throughput.html`, `ResponseTimes.html`, plus `CustomsGraphs.html` and `locust_report.html` when present) loads in an isolated 1440 px-wide `<iframe>` (`deviceScaleFactor: 2`). Expanding `.collapse.portlet-content` via DOM/CSS and invoking all 14 JMeter `refresh*` graph initializers directly avoids Bootstrap hash-navigation interference.
- **Exact 1-Page-Per-Tab CSS Sizing:** Empty tabs such as `CustomsGraphs.html` are automatically pruned, yielding a crisp 4-page `jmeter_report.pdf` where each tab occupies an exact dynamic CSS `@page` height (`size: 1440px <measuredH>px`) without split charts.

7 Empirical Verification Results

7.1 Full Five-Tier Capacity Campaign (`https://najealp.top`)

On November 11, 2026 (17:41–17:45 local time, `jmeter_report.pdf` and `jmeter_baseline.json`), following database initialization with `test/load/hell.sql` and deployment of the Tier-3 application/Nginx optimizations in Section 1, the standalone five-tier JMeter suite executed cleanly against `https://najealp.top` under full `config.json` pacing ($V \in \{100, 250, 500, 750, 1000\}$, $D_{\text{tier}} = 45$ s, $R_{\text{ramp}} = 30$ s, $T_{\text{think}} \sim \mathcal{U}(1500, 4000)$ ms). Across 23,157 **total requests**, the platform recorded **zero failures (**0.00%**), zero `401` rejections, zero `403` errors, and zero `5xx` responses**.

Table 2 summarizes the Phase 1 gates and all five stepped tiers alongside aggregate load-dashboard totals (`statistics.json` / `jmeter_report.pdf`) and two historical reference baselines preserved for contrast (Nov 10 unpaced Node read probe; Nov 11 10:48 pre-hardening Locust write/read run).

Table 2: Verified telemetry across all JMeter campaign stages (`17:41-17:44` full run) and earlier reference baselines targeting `https://najealp.top`.

Execution Stage / Tier	Reqs	Errs / 401	RPS	Mbps	APDEX	Avg	p_{50}	p_{95}	p_{99} / Max
Phase 1: JMeter Setup Auth (`9.72 s`)	5	0/0	0.51	—	—	1941.6	81.0	9014.0	9014.0 / 9014
Phase 1: Coverage & CRUD (`17.87 s`)	186	0/0	10.41	—	—	89.98	80.0	158.0	349.0 / 360
Tier 1 (`100 VUs`, 43.30 s)	888	0/0	20.51	1.07	0.999	96.38	72.0	248.0	360.0 / 618
Tier 2 (`250 VUs`, 43.20 s)	2,275	0/0	52.66	2.70	0.999	94.69	71.0	231.0	328.0 / 646
Tier 3 (`500 VUs`, 43.70 s)	4,488	0/0	102.69	5.32	0.999	90.96	70.0	220.0	300.0 / 1611
Tier 4 (`750 VUs`, 44.37 s)	6,741	0/0	151.92	7.86	0.998	93.44	70.0	220.0	309.0 / 1290
Tier 5 (`1000 VUs`, 44.45 s)	8,574	0/0	192.90	10.07	0.923	281.52	117.0	959.0	2770.0 / 12491
Aggregate Load Total (226.71 s)*	22,966	0/0	101.30	5.14	0.970	163.41	73.00	566.00	1577.93 / 12491
Historical Pre-Tier-3 References (Distinct Earlier Test Windows, Included for Context):									
Ref: Node Unpaced (`10 VUs`, 5 s, Nov 10)	473	0/0	93.18	16.74	—	105.62	89.4	170.8	430.0 / 523
Ref: Node Unpaced (`25 VUs`, 5 s, Nov 10)	1,066	0/0	207.25	35.78	—	118.28	105.4	200.3	382.4 / 498
Ref: Locust Write Mix (`10:48` Pre-Hardening)	5,429	466 † /0	—	—	—	—	—	—	—

Latency columns are in milliseconds. *HTML/PDF dashboard (`jmeter_report.pdf`) reports 641.94 KB/s recv (5.14 Mbps avg), 19.75 KB/s sent, min = 44 ms, and Commons Math interpolated percentiles; individual tier rows report discrete ceiling percentiles from `jmeter_baseline.json`. †Client-side TLS/EOF socket errors during pre-Tier-3 write/sync-bcrypt testing; 0 HTTP `401` and 0 HTTP `5xx` server errors.

HTML Dashboard APDEX & Response-Time Distribution (`jmeter_report.pdf`)

Across all 22,966 load-phase transactions (22,964 HTTP `200` + 2 HTTP `304` conditional cache hits), the platform achieved an aggregate **APDEX** = 0.970 ($T = 500$ ms, $F = 1500$ ms):

- **Satisfied ($t \leq 500$ ms):** 21,832 requests (95.06%)
- **Tolerating ($500 < t \leq 1500$ ms):** 913 requests (3.98%)
- **Frustrated ($t > 1500$ ms):** 221 requests (0.96%), of which 220 occurred during Tier 5 (1,000 VUs) and 1 during Tier 3 (500 VUs)
- **Errors:** 0 requests (0.00%); per-route APDEX ranged cleanly from 0.932 to 0.999.

Validation Against Little's Law: Substituting $\mathbb{E}[T_{\text{think}}] = 2.75$ s into Eq. (3), full-concurrency steady-state ceilings span 35.1 req/s (100 VUs), 87.9 req/s (250 VUs), 176.0 req/s (500 VUs), 263.8 req/s (750 VUs), and 329.9 req/s (1,000 VUs). Because each 45 s tier spends 30 s linearly ramping threads from $0 \to V$, theoretical ramp-weighted mean concurrency equals $\frac{30/2+15}{45} \approx 66.7\%$ of V (yielding $\approx$ 58–60% of the steady ceiling across the 43.2–44.5 s active sample span). Observed tier averages (20.51, 52.66, 102.69, 151.92, and 192.90 req/s) track this exact ramp factor, while post-ramp full-1,000-VU 1-second plateaus in `jmeter_results_load.jtl` hit 252–312 **req/s** (291 req/s peak in JMeter's HTML throughput plot), agreeing closely with Eq. (3).

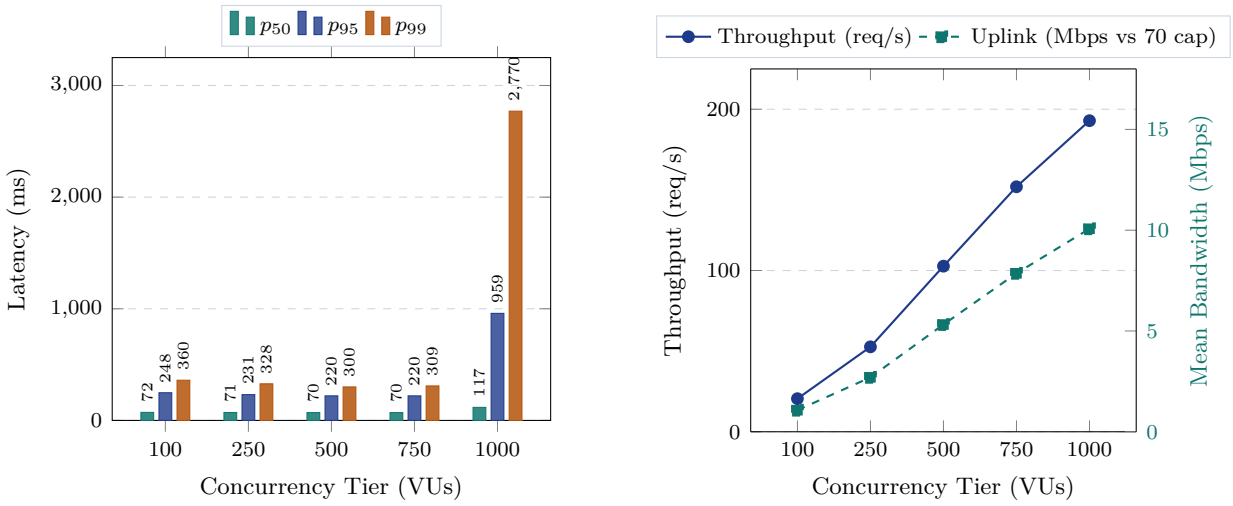


Figure 2: Five-tier scaling behavior (`100-1000 VUs`). **Left:** Latency percentiles stay flat through 750 VUs ($p_{50} \leq 72$ ms, $p_{99} \leq 360$ ms) before inflecting at 1,000 VUs. **Right:** Mean throughput and bandwidth scale smoothly to 192.90 req/s (10.07 Mbps, just 14.4% of the 70 Mbps physical link).

7.2 Route-Level Performance Profile and Tier-5 Inflection Diagnosis

Inspecting per-route metrics across all 51 load labels in `jmeter_report.pdf` (`statistics.json` and `dashboard.js`) paired with raw `jmeter_results_load.jtl` timestamp/connect slicing provides three crisp architectural insights. Table 3 reports the highest-p_{95} endpoints alongside fast cached reference/admin endpoints.

Table 3: Representative route telemetry ordered by p_{95} (`jmeter_report.pdf` / `statistics.json`, $n = 22{,}966$ load samples).

Endpoint Label (Weight Bucket)	Calls	KB/req	APDEX	p_{50}	p_{95}	p_{99}	Max
`GET /api/events?_include=...&_page=1` (B0)	894	14.35	0.932	141.0	1003.3	3043.6	5211
`GET /api/events?_sort=eventDate&_order=asc` (B0)	940	14.60	0.932	140.0	994.9	2814.0	6266
`GET /api/events?_page=1&_per_page=10` (B0)	881	14.35	0.946	140.0	820.6	1944.8	9894
`GET /api/bulletins?_sort=bulletinDate&_order=desc` (B0)	901	12.19	0.960	72.0	812.3	2611.4	5700
`GET /api/bulletins?_page=1&_per_page=10` (B0)	896	13.39	0.958	71.0	787.4	3227.5	9776
`GET /images/engineer.png` (Static Asset, B0)	905	13.10	0.959	70.0	779.8	2387.1	11986
`GET /api/admin/bulletins?_page=1&_per_page=10` (B3)	161	13.33	0.963	74.0	739.5	2836.1	3024
`GET /api/comments?_page=1&_per_page=10` (B1)	339	11.66	0.954	73.0	724.0	3141.0	7130
`GET /api/userConnections?_include=...` (B1)	353	11.03	0.963	75.0	703.3	1682.7	12491
`GET /api/bulletins?_include=...&_page=1` (B0)	940	13.79	0.968	72.0	688.5	1618.3	5638
Representative Fast / Cached Routes in the Same Run (Comparison, latency in ms):							
`GET /api/admin/dashboard-stats` (B3, 20s Cache)	149	5.56	0.977	72.0	463.5	1026.0	1136
`GET /api/users?_where=active&_include` (B1)	338	3.80	0.979	73.0	312.4	810.0	4994
`GET /api/auth/me [Regular]` (B1)	349	0.87	0.999	71.0	233.0	407.0	501
`GET /api/contentStatuses` (B2, 300s Cache)	255	0.57	0.988	69.0	210.2	1282.2	1800

1. **Flat Sub-360ms Scaling Through 750 VUs & Bandwidth Validation:** Across Tiers 1 through 4 (100–750 VUs, 14,392 samples), median latency actually tightened slightly (72.0 → 70.0 ms) and p_{95} held inside 220–248 ms with only 1 sample above 1500 ms. Average paginated event and bulletin payloads dropped to 12.19–14.60 KB/req (6.34 KB/req blended across all 51 routes), limiting Tier 5 network consumption to 10.07 Mbps mean (14.39% of 70 Mbps; 15.46 Mbps / 22.1% per-second JTL peak)—confirming zero network-link saturation.

2. **Dual Median RTT Strata (69–75 ms vs. 140–144.5 ms):** Static images, status dictionaries, auth

token validations, `/api/bulletins`, and `/api/users?_where=active&_include` ($p_{95} = 312.4$ ms vs. 1935 ms pre-Tier-3) all cluster right on the ≈ 70 ms single-RTT WAN latency floor. By contrast, all paginated `/api/events` and `/api/admin/events` samplers settle cleanly one ≈ 70 ms step higher at $p_{50} = 140.0$–144.5 ms.

3. **Root Cause of Tier 5 Tail Excursion — Transient TLS/Accept Queueing:** Decomposing Tier 5 (1,000 VUs, $n = 8{,}574$) by JTL `Connect` times and 1-second windows proves the 959 ms p_{95} and 12,491 ms `Max` did *not* originate from steady-state SQLite read contention:
   - Warm keep-alive requests (`Connect = 0`, $n = 7{,}574$) averaged 257.93 ms ($p_{50} = 97$ ms, $p_{95} = 895$ ms). New TLS handshakes (`Connect > 0`, $n = 1{,}000$) averaged 460.23 ms ($p_{50} = 228$ ms, $p_{95} = 1548$ ms), driven by TCP/TLS setup jumping to $\bar{C} = 272.7$ ms ($p_{95} = 1{,}126$ ms, max = 9,831 ms) compared to 140.9–146.3 ms mean across Tiers 1–4.
   - Second-by-second slicing pinpoints a transient 9-second window (`t+206s..t+214s`, i.e. +24–32 s into Tier 5 as the final ramp cohort pushed active throughput to 252–284 req/s) absorbing the transient $p_{50} = 318$–590 ms / $p_{95} = 1{,}165$–$3{,}299$ ms queue. Tellingly, even static `GET /images/engineer.png` (bypassing Node and SQLite via Nginx `try_files`) hit max = 11,986 ms in that exact `t+208s` window. From `t+216s` through `t+225s` at full 1,000-VU concurrency (248–312 req/s), latency recovered cleanly ($p_{50} = 85$–155 ms except one 233 ms tick; $p_{95} = 261$–679 ms; 0 errors).
4. **Workload Scope Boundary:** Because the `17:41` campaign executed standalone `run_jmeter.ps1` without `-IncludeWritesInLoad` or concurrent Locust writes (the 31 mutating `201/204` requests ran sequentially in Phase 1 Coverage), its 0.00% error rate and 0.970 APDEX certify sustained **1,000-VU read/cache/TLS capacity**, while concurrent write serialization under `DELETE` mode continues to be validated via joint `run_parallel_suite.ps1` runs.

8 Operational Governance and Reproducibility

8.1 Database Hygiene, Snapshot Recovery, and Rate-Limiter Controls

Pre-test normalization and post-test cleanup form a strict operational bracket around every campaign against `dev.db`:

- **Pre-Test Backup & Provisioning (`test/load/hell.sql`):** Capture `cp dev.db dev.db.bak_pre_stress_$(date +%s)` before executing `hell.sql`, especially because Step 6 resets all existing `UserAuth` and `Admin` hashes for staging benchmark access.
- **Post-Test Teardown (`test/load/teardown.sql`):** Automated API `DELETE` cleanup hooks purge transient Phase 1 and Locust entities; `teardown.sql` removes remaining synthetic `loadtest_%@najealp.top`, `test.official%`, and `test.regular%` records followed by `VACUUM`, or restores original user credentials directly via `cp dev.db.bak_pre_stress_* dev.db`.
- **Rate-Limit Restoration:** Operators moving from benchmark mode back to public production should confirm whether `authLimiter` and `contentLimiter` in `api/index.js` transition from `skip: () => true` back to active throttling.

8.2 Standardized Execution Profiles

Reproducing either the fast DryRun gate, standalone 5-tier JMeter certification (`jmeter_report.pdf`), or unified dual-engine write/read testing requires a single PowerShell command from the repository root (with Apache JMeter 5.6.3 and Edge/Chrome installed):

Listing 3: Reproducible invocation profiles in `test/load/`.

```
# 1. Fast pre-flight verification & DryRun ([10, 25] VUs x 10s + 20 Locust VUs)
pwsh -NoProfile -ExecutionPolicy Bypass -File test\load\run_parallel_suite.ps1
    -DryRun

# 2. Standalone 5-tier JMeter capacity run (reproduces jmeter_report.pdf & .xlsx)
pwsh -NoProfile -ExecutionPolicy Bypass -File test\load\run_jmeter.ps1 -AllReports

# 3. Full joint 5-tier concurrency + Locust write contention suite
pwsh -NoProfile -ExecutionPolicy Bypass -File test\load\run_parallel_suite.ps1
```

Architectural Status & Next-Stage Recommendations

1. **Tier-3 Hardening Empirically Validated:** Switching `bcryptjs` to async `await bcrypt.hash/compare`, slimming `defaultListIncludes`, enabling `feedCache`/`lookupCache`/`adminDashboardStatsCache`, adding 33 Prisma indexes, and aligning Nginx + Node keep-alive lifted clean multi-hundred-VU capacity to 750 **VUs at APDEX** = 0.998 and 1,000 **VUs with** 0.00% **HTTP errors (APDEX** = 0.970).
2. **Smooth** 1,000**-VU Ramp Accept Queues & Event Feed Latency:** Profile OS/Nginx socket listen backlog (`somaxconn`, Nginx `listen ... backlog`, TLS session resumption) to flatten the +24–32 s TLS handshake spike during 750 → 1,000-VU ramps, and inspect why `/api/events` paginated routes sit at $p_{50} = 140$ ms ($\approx 2\times$ `/api/bulletins`'s 71 ms).
3. **Preserve `DELETE`-Mode Boundaries via Joint Runs:** Continue honoring `PRAGMA journal_mode = DELETE` alongside `busy_timeout = 10000`, and re-run `run_parallel_suite.ps1` with concurrent Locust writes after proxy/event-route tuning to certify mixed read/write tail bounds across 500–1,000 VUs.